

Printing, Integers

1. Printing to the Console (fmt package)

To print data to the screen, Go uses a built-in package called **fmt** (short for format).

The most powerful printing tool is **fmt.Printf** (Print Formatted). It allows you to inject variables directly into a string using "conversion characters" (format verbs).

- **How it works:** You write a string, and wherever you want a variable to appear, you put a % symbol followed by a letter indicating the type. For example, %s is used for injecting a string.
- **Example:** `fmt.Printf("Hi %s", "Joe")` will output: **Hi Joe**

3. Integers: Sizes and Signs

When you just say `int`, the Go compiler automatically decides how much memory to allocate based on your computer (usually **32** or **64** bits). However, Go gives you exact control over memory if you need it.

- **Explicit Sizes:** You can declare exactly how many bits of memory an integer should use: **int8**, **int16**, **int32**, or **int64**.
- **Signed vs. Unsigned (uint):** A standard integer (like **int8**) uses its first bit (the most significant bit) to track the **sign** (whether the number is positive or negative).
 - An **unsigned** integer (like **uint8**) drops the negative capability entirely. It uses that extra bit to double its maximum positive value.
 - **Example:** An **int8** can store num from **-128 to +127**. A **uint8** can store num from **0 to 255**.

Expert Takeaway:

As a developer, you might wonder why you would ever bother specifying an **int8** or **uint16** instead of just using a standard `int`. The answer is **Network and Database Optimization**.

If you are building a microservice that sends millions of real-time user status updates (where a status is just a number between 1 and 5), using an **int8** instead of a **64-bit** `int` reduces your memory usage and network payload size by **87%**! Go gives you the power to be that precise.